Capstone Project on Flipkart

Project Objective

- The project objective was to collect real-world data; I have taken a global e-commerce brand Flipkart. I have to do some cleaning on the data like renaming columns, handling missing values etc. Then created a recommendation model out of it.

PHASE 1: WEB SCRAPING

[2]:
```python
#importing all the necessary libaries
import requests
from bs4 import BeautifulSoup
import pandas as pd
import numpy as np
import time
import random
from urllib.parse import urlparse, parse_qs, urlencode, urlunparse
```

[3]:
```python
def scrape_flipkart_fashion(base_url, max_records=1200):

    # Import again inside function to ensure availability
    from urllib.parse import urlparse, parse_qs, urlencode, urlunparse

    products = []
    headers = {
        'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36',
        'Accept-Language': 'en-US,en;q=0.9',
        'Accept-Encoding': 'gzip, deflate, br',
        'Accept': 'text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8',
        'Connection': 'keep-alive',
        'Upgrade-Insecure-Requests': '1'
    }

    page = 1
    consecutive_failures = 0
    max_failures = 3

    print(f"Starting to scrape: {base_url}")
    print(f"Target: {max_records} records")
    print("=" * 70)

    while len(products) < max_records:
        try:
            # Parse the URL and add page parameter
            parsed_url = urlparse(base_url)
            query_params = parse_qs(parsed_url.query)
```

```python
            # Add or update page parameter
            query_params['page'] = [str(page)]

            # Reconstruct URL with page parameter
            new_query = urlencode(query_params, doseq=True)
            page_url = urlunparse((
                parsed_url.scheme,
                parsed_url.netloc,
                parsed_url.path,
                parsed_url.params,
                new_query,
                parsed_url.fragment
            ))

            print(f"\n Scraping Page {page}...")
            print(f"Current count: {len(products)}/{max_records}")

            # Add random delay to avoid blocking
            time.sleep(random.uniform(2, 4))

            # Make request
            response = requests.get(page_url, headers=headers, timeout=15)

            if response.status_code != 200:
                print(f"  Failed to fetch page {page}. Status: {response.status_code}")
                consecutive_failures += 1
                if consecutive_failures >= max_failures:
                    print(f" Too many failures. Stopping.")
                    break
                continue

            # Reset failure counter on success
            consecutive_failures = 0

            # Parse HTML
            soup = BeautifulSoup(response.content, 'html.parser')

            # Find product containers using the correct parent element
            # Products are contained in divs that have the product name links
```

```python
    # Products are contained in divs that have the product name links
    product_links = soup.find_all('a', {'class': 'atJtCj'})

    if not product_links:
        # Try alternative class name
        product_links = soup.find_all('a', {'class': 'atJtCj Qum9aC'})

    # Get parent containers for each product
    product_cards = []
    for link in product_links:
        # Get the parent container that has all product info
        parent = link.find_parent('div', class_=lambda x: x and ('tUxRFH' in str(x) or 'CGtC98' in str(x) or 'slAVV4' in str(x)))
        if not parent:
            parent = link.find_parent('div')
        if parent and parent not in product_cards:
            product_cards.append(parent)

    if not product_cards or len(product_cards) == 0:
        print(f"  No products found on page {page}. Might have reached the end.")
        consecutive_failures += 1
        if consecutive_failures >= max_failures:
            print("Stopping - no more products available.")
            break
        page += 1
        continue

    print(f"   Found {len(product_cards)} product cards on this page")

    # Extract data from each product card
    page_products = 0
    for card in product_cards:
        if len(products) >= max_records:
            break

        try:
            product = {}

            # Product Name - using exact class names you provided
            name_tag = card.find('a', {'class': 'atJtCj Qum9aC'})
```

```python
    # Product Name - using exact class names you provided
    name_tag = card.find('a', {'class': 'atJtCj Qum9aC'})
    if not name_tag:
        name_tag = card.find('a', {'class': 'atJtCj'})

    product['name'] = name_tag.text.strip() if name_tag else 'N/A'

    # Skip if no name found
    if product['name'] == 'N/A' or not product['name']:
        continue

    # Company/Brand
    company_tag = card.find('div', {'class': 'Fo1I0b'})
    product['company'] = company_tag.text.strip() if company_tag else 'N/A'

    # Current Price
    price_tag = card.find('div', {'class': 'hZ3P6w'})
    product['price'] = price_tag.text.strip() if price_tag else 'N/A'

    # Original Price
    original_price_tag = card.find('div', {'class': 'kRYCnD'})
    product['original_price'] = original_price_tag.text.strip() if original_price_tag else 'N/A'

    # Discount
    discount_tag = card.find('div', {'class': 'HQe8jr'})
    product['discount'] = discount_tag.text.strip() if discount_tag else 'N/A'

    # Special Offers (Buy 2, Hot Deal, Only Few Left, etc.)
    offers_tag = card.find('div', {'class': 'HZ0E6r Rm9_cy'})
    if not offers_tag:
        offers_tag = card.find('div', {'class': 'HZ0E6r'})
    product['offers'] = offers_tag.text.strip() if offers_tag else 'N/A'

    # Size
    size_tag = card.find('div', {'class': 'FD7OvP'})
    product['size'] = size_tag.text.strip() if size_tag else 'N/A'
```

```python
                # Product URL
                if name_tag and 'href' in name_tag.attrs:
                    href = name_tag['href']
                    if href.startswith('/'):
                        product['url'] = 'https://www.flipkart.com' + href
                    else:
                        product['url'] = href
                else:
                    product['url'] = 'N/A'

                # Page number
                product['page'] = page

                # Add to products list
                products.append(product)
                page_products += 1

            except Exception as e:
                # Skip problematic products
                continue

        print(f"✓ Extracted {page_products} products from page {page}")
        print(f"   Total so far: {len(products)}/{max_records}")

        # Move to next page
        page += 1

    except Exception as e:
        print(f" Error on page {page}: {e}")
        consecutive_failures += 1
        if consecutive_failures >= max_failures:
            print("Too many consecutive errors. Stopping.")
            break
        page += 1
        continue

    return products

print(" Scraping function defined successfully!")
```

```
✓ Scraping function defined successfully!
```

[4]:
```python
flipkart_url = "https://www.flipkart.com/clothing-and-accessories/topwear/pr?sid=clo,ash&p[]=facets.ideal_for%255B%255D%3DMen&p[]=facets.ideal_for%255B%

print("🚀 Starting Flipkart Fashion scraper...")
print("Category: Men's Topwear")
print("Note: This may take 10-15 minutes to extract 1200 records")
print("=" * 70)

# Run the scraper
scraped_data = scrape_flipkart_fashion(flipkart_url, max_records=1200)

# Convert to DataFrame
df = pd.DataFrame(scraped_data)

# Display summary
print("\n" + "=" * 70)
print(f"✓ SCRAPING COMPLETED!")
print(f"Total records extracted: {len(df)}")
print("=" * 70)
```

```
----------------------------------------------------------------------
Starting to scrape: https://www.flipkart.com/clothing-and-accessories/topwear/pr?sid=clo,ash&p[]=facets.ideal_for%255B%255D%3DMen&p[]=facets.ideal_fo
r%255B%255D%3Dmen&otracker=categorytree&fm=neo%2Fmerchandising&iid=M_cf6958ea-8f21-444b-a07c-1321c3aabf1a_1_X1NCR146KC29_MC.RLI1MOY42WPG&cid=RLI1MOY4
2WPG
Target: 1200 records
======================================================================

📄 Scraping Page 1...
Current count: 0/1200
   Found 40 product cards on this page
✓ Extracted 40 products from page 1
   Total so far: 40/1200

📄 Scraping Page 2...
Current count: 40/1200
   Found 40 product cards on this page
✓ Extracted 40 products from page 2
   Total so far: 80/1200
```

```
[10]: # Convert to DataFrame
      df = pd.DataFrame(scraped_data)
```

```
•[11]: # Save to CSV
       csv_filename = 'flipkart_mens_topwear_1000_records.csv'
       df.to_csv(csv_filename, index=False, encoding='utf-8-sig')
       print(f"✓ CSV saved: {csv_filename}")

       ✓ CSV saved: flipkart_mens_topwear_1200_records.csv
```

1. DATA COLLECTION METHORD

- I have used BeautifulSoup from bs4 and requests, and scraped the section of t-shirt from the Flipkart website and successfully scraped 1000 records.

PHASE 2 : DATA CLEANING & PREPROCESSING

PHASE 2 : DATA CLEANING & PREPROCESSING

```
[11]: df.columns
```

```
[11]: Index(['name', 'company', 'price', 'original_price', 'discount', 'offers',
       'size', 'url', 'page'],
      dtype='str')
```

```
[12]: df['price'] = (
          df['price']
          .str.replace(r'[^\d.]', '', regex=True)
          .astype(float)
      )
```

```
[13]: df['original_price'] = (
          df['original_price']
          .str.replace(r'[^\d.]', '', regex=True)
          .astype(float)
      )
```

```
[14]: df['original_price'] = pd.to_numeric(
          df['original_price'].astype(str).str.replace(r'[^\d.]', '', regex=True),
          errors='coerce'
      )
```

```
[15]: df['discount'] = (
          df['discount']
          .str.extract(r'(\d+)')
          .astype(float)
      )
```

[16]:
```
df.head()
```

[16]:

	name	company	price	original_price	discount	offers	size	url	page
0	Men Solid Zip Neck Cotton Blend Black T-Shirt	KAJARU	298.0	999.0	70.0	Hot Deal	NaN	https://www.flipkart.com/kajaru-solid-men-zip-...	1
1	Men Printed Polo Neck Cotton Blend Black, Grey...	TRIPR	338.0	999.0	66.0	Buy 2 items, save extra 5%	NaN	https://www.flipkart.com/tripr-printed-men-pol...	1
2	Pack of 4 Men Solid Round Neck Cotton Blend Mu...	TRIPR	459.0	1999.0	77.0	Buy 2 items, save extra 5%	NaN	https://www.flipkart.com/tripr-solid-men-round...	1
3	Men Striped Zip Neck Cotton Blend Black, White...	Try This	306.0	999.0	69.0	Only few left	NaN	https://www.flipkart.com/try-striped-men-zip-n...	1
4	Men Solid Henley Neck Cotton Blend Black T-Shirt	TRIPR	192.0	1999.0	90.0	Hot Deal	NaN	https://www.flipkart.com/tripr-solid-men-henle...	1

[17]:
```
df.isnull().sum()
```

[17]:
```
name                0
company             0
price               0
original_price      6
discount            9
offers             49
size             1000
url                 0
page                0
dtype: int64
```

[18]:
```
df['original_price'] = df['original_price'].fillna(df['original_price'].median())
```

[19]:
```
df['discount'] = df['discount'].fillna(0)
```

[20]:
```
df['offers'] = df['offers'].fillna("No Offer")
```

[21]:
```
df.isnull().sum()
```

[21]:
```
name                0
company             0
price               0
original_price      0
discount            0
offers              0
size             1000
url                 0
page                0
dtype: int64
```

[23]:
```
df.info()
df.head()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   name            1000 non-null   str
 1   company         1000 non-null   str
 2   price           1000 non-null   float64
 3   original_price  1000 non-null   float64
 4   discount        1000 non-null   float64
 5   offers          1000 non-null   str
 6   size            0 non-null      float64
 7   url             1000 non-null   str
 8   page            1000 non-null   int64
dtypes: float64(4), int64(1), str(4)
memory usage: 485.2 KB
```

[23]:

	name	company	price	original_price	discount	offers	size	url	page
0	Men Solid Zip Neck Cotton Blend Black T-Shirt	KAJARU	298.0	999.0	70.0	Hot Deal	NaN	https://www.flipkart.com/kajaru-solid-men-zip-...	1
1	Men Printed Polo Neck Cotton Blend Black, Grey...	TRIPR	338.0	999.0	66.0	Buy 2 items, save extra 5%	NaN	https://www.flipkart.com/tripr-printed-men-pol...	1
2	Pack of 4 Men Solid Round Neck Cotton Blend Mu...	TRIPR	459.0	1999.0	77.0	Buy 2 items, save extra 5%	NaN	https://www.flipkart.com/tripr-solid-men-round...	1
3	Men Striped Zip Neck Cotton Blend Black, White...	Try This	306.0	999.0	69.0	Only few left	NaN	https://www.flipkart.com/try-striped-men-zip-n...	1

- Create useful features (rating %, price band, category, etc.)

[24]:
```
df['discount_pct'] = ((df['original_price'] - df['price']) / df['original_price']) * 100
df['discount_pct'] = df['discount_pct'].round(2)
```

[25]:
```
df.discount_pct
```

[25]:
```
0      70.17
1      66.17
2      77.04
3      69.37
4      90.40
       ...
995    69.98
996    87.84
997    82.02
998    67.47
999    77.09
Name: discount_pct, Length: 1000, dtype: float64
```

[26]:
```
df['savings'] = df['original_price'] - df['price']
df.savings
```

[26]:
```
0       701.0
1       661.0
2      1540.0
3       693.0
4      1807.0
        ...
995     909.0
996    1756.0
997    2870.0
998     674.0
999     693.0
Name: savings, Length: 1000, dtype: float64
```

[27]:
```
def price_band(p):
    if p < 500:
        return "Budget"
    elif 1500 <= p:
        return "Mid-Range"
    else:
        return "Premium"

df['price_band'] = df['price'].apply(price_band)
df.price_band
```

[27]:
```
0       Budget
1       Budget
2       Budget
3       Budget
4       Budget
        ...
995     Budget
996     Budget
997    Premium
998     Budget
999     Budget
Name: price_band, Length: 1000, dtype: str
```

[28]:
```
df.price_band.unique()
```

[28]:
```
<ArrowStringArray>
['Budget', 'Premium']
Length: 2, dtype: str
```

[29]:
```
df[df['price_band']=='Mid-Range']
```

[29]:

name	company	price	original_price	discount	offers	size	url	page	discount_pct	savings	price_band

[30]:
```
df['high_discount'] = df['discount_pct'].apply(lambda x: 1 if x >= 50 else 0)
df.high_discount
```

[30]:
```
0      1
1      1
2      1
3      1
4      1
```

```
[32]: df[df['high_discount']== 0]
```

[32]:

	name	company	price	original_price	discount	offers	size	url	page	discount_pct	savings	price_band	high_discou
5	Men Solid Round Neck Polyester Dark Blue T-Shirt	JOCKEY	998.0	999.0	0.0	Only few left	NaN	https://www.flipkart.com/jockey-solid-men-roun...	1	0.10	1.0	Premium	
5	Men Solid Round Neck Polyester Dark Blue T-Shirt	JOCKEY	998.0	999.0	0.0	Only few left	NaN	https://www.flipkart.com/jockey-solid-men-roun...	4	0.10	1.0	Premium	
	Pack of 1 Pace												

```
[33]: df.head()
      df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 13 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   name            1000 non-null   str
 1   company         1000 non-null   str
 2   price           1000 non-null   float64
 3   original_price  1000 non-null   float64
 4   discount        1000 non-null   float64
 5   offers          1000 non-null   str
 6   size            0 non-null      float64
 7   url             1000 non-null   str
 8   page            1000 non-null   int64
 9   discount_pct    1000 non-null   float64
 10  savings         1000 non-null   float64
 11  price_band      1000 non-null   str
 12  high_discount   1000 non-null   int64
dtypes: float64(6), int64(2), str(5)
memory usage: 522.4 KB
```

```
[34]: df.drop(columns=['size'], inplace=True)
```

```
[35]: df.columns
```

```
[35]: Index(['name', 'company', 'price', 'original_price', 'discount', 'offers',
       'url', 'page', 'discount_pct', 'savings', 'price_band',
       'high_discount'],
      dtype='str')
```

```
[36]: df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   name            1000 non-null   str
 1   company         1000 non-null   str
 2   price           1000 non-null   float64
 3   original_price  1000 non-null   float64
 4   discount        1000 non-null   float64
 5   offers          1000 non-null   str
 6   url             1000 non-null   str
 7   page            1000 non-null   int64
 8   discount_pct    1000 non-null   float64
 9   savings         1000 non-null   float64
 10  price_band      1000 non-null   str
 11  high_discount   1000 non-null   int64
dtypes: float64(5), int64(2), str(5)
memory usage: 514.6 KB
```

2. Cleaning Steps

- I have used pandas and Numpay to clean the data, handling missing values, renaming columns and made it ready for analysis.

PHASE 3 : EXPLORATORY DATA ANALYSIS (EDA)

PHASE 3 : EXPLORATORY DATA ANALYSIS (EDA)

- **Summary statistics (mean, median, spread)**

[37]:
```
df[['price', 'original_price', 'discount_pct', 'savings']].describe()
```

[37]:

	price	original_price	discount_pct	savings
count	1000.000000	1000.000000	1000.000000	1000.000000
mean	346.805000	1559.716000	75.275500	1212.911000
std	148.020522	659.137841	11.495377	608.322388
min	104.000000	351.000000	0.070000	1.000000
25%	246.000000	999.000000	69.470000	713.500000
50%	310.000000	1499.000000	76.240000	1130.000000
75%	406.500000	1999.000000	83.285000	1685.500000
max	1398.000000	4299.000000	93.090000	3547.000000

[38]:
```
df['price'].mean()
```

[38]: 346.805

[39]:
```
df['price'].median()
```

[39]: 310.0

[40]:
```
df['price'].std()
```

[40]: 148.02052239486946

[41]:
```
df['price'].var()
```

[41]: 21910.075050050047

[42]:
```
df['price'].max() - df['price'].min()
```

[42]: 1294.0

- **Trend analysis (time, category, rating, price, etc.)**

[43]:
```
df.groupby('company')['price'].mean().sort_values(ascending=False)
```

[43]:
```
company
Snitch              1398.000000
WROGN               1123.333333
U.S. POLO ASSN.     1064.000000
JOCKEY              1008.500000
LOUIS PHILIPPE       884.000000
                        ...
Mundali              147.000000
PRABHU               142.000000
FUBACK               138.000000
Riva                 112.000000
Axxitude             106.000000
Name: price, Length: 190, dtype: float64
```

[44]:
```
df.company.unique()
```

[44]:
```
<ArrowStringArray>
[                    'KAJARU',                        'TRIPR',
                  'Try This',                        'Surhi',
                    'VeBNoR',                      'N AND J',
                 'JACKBELLA',                        'DIMMY',
                   'Raymond',                        'WOXEN',
 ...
                 'Southcity',                  'Tee Gallery',
                      'DBAX',                      'GLAUCUS',
 'United Colors of Benetton',                   'BLUE SQUAD',
                'PrintHolic',                     'Lux Nitro',
                'hkCreation',                     'BlueFicus']
Length: 190, dtype: str
```

[45]:
```
df.groupby('company')['discount_pct'].mean().sort_values(ascending=False)
```

[45]:
```
company
FUBACK          93.090000
Surhi           89.490000
Smosh           89.440000
Voroxy          88.967500
HKV FASHION     88.831667
                   ...
Maxzone         30.570000
Lux Nitro       17.090000
```

[46]:
```
company_discount_pact = df.groupby('company')['discount_pct'].mean().sort_values(ascending=False)
```

[47]:
```
df['company'].value_counts()
```

[47]:
```
company
N AND J                     76
TRIPR                       69
METRONAUT                   50
VeBNoR                      40
RED TAPE                    39
                            ..
United Colors of Benetton    1
PrintHolic                   1
Lux Nitro                    1
hkCreation                   1
BlueFicus                    1
Name: count, Length: 190, dtype: int64
```

[48]:
```
page_trend = df.groupby('page')[['price','discount_pct']].mean()
page_trend.head()
```

[48]:

page	price	discount_pct
1	333.675	76.64050
2	318.450	77.62675
3	322.375	76.12625
4	357.925	74.14900
5	316.625	75.94475

```
[49]: df['name'].value_counts().head(10)
```

```
[49]: name
Men Regular Fit Solid Casual Shirt                                50
Men Regular Fit Solid Spread Collar Casual Shirt                  46
Men Slim Fit Self Design Spread Collar Casual Shirt               39
Men Regular Fit Solid Button Down Collar Casual Shirt             33
Men Regular Fit Solid Button Down Collar Formal Shirt             26
Pack of 4 Men Solid Round Neck Cotton Blend Multicolor ...        25
Men Solid Polo Neck Polyester Navy Blue T-Shirt                   25
Pack of 3 Men Solid Round Neck Cotton Blend Multicolor ...        25
Pack of 4 Men Solid Round Neck Polyester Silver, Blue, ...        24
Men Solid Polo Neck Polyester Blue T-Shirt                        23
Name: count, dtype: int64
```

```
[50]: df.groupby('company')['savings'].mean().sort_values(ascending=False).head(10)
```

```
[50]: company
RED TAPE         3013.410256
Yazole           2287.250000
London Hills     2281.000000
Allen Cooper     2220.000000
Coofandy         2114.000000
InkTees          2107.000000
Helken           2107.000000
Think Tech       2080.000000
Voroxy           1956.500000
FUBACK           1858.000000
Name: savings, dtype: float64
```

```
[51]: company_savings = df.groupby('company')['savings'].mean().sort_values(ascending=False).head(10)
```

```
[52]: df['high_discount'].value_counts()
```

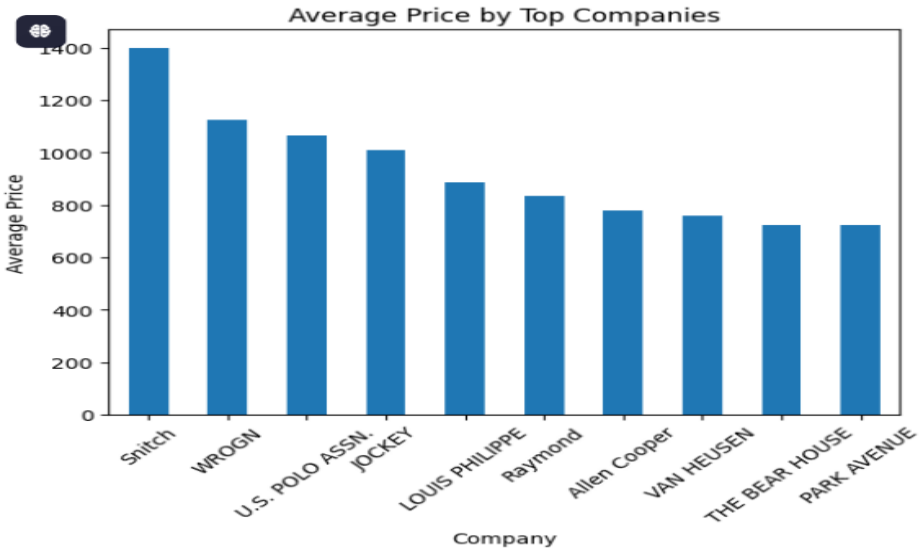
```
[52]: high_discount
1    980
0     20
Name: count, dtype: int64
```

```
[53]: df['price_band'].value_counts()
```

```
[53]: price_band
Budget      910
Premium      90
Name: count, dtype: int64
```

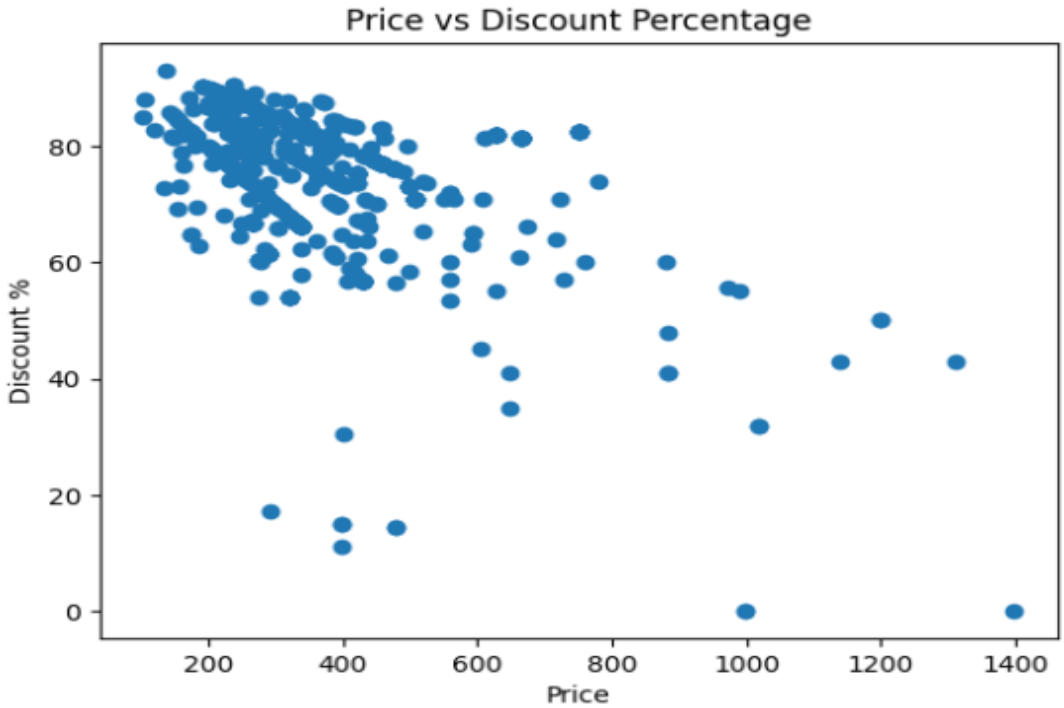
- **Visualizations:**

```
[54]: import matplotlib.pyplot as plt

top_companies = df.groupby('company')['price'].mean().sort_values(ascending=False).head(10)

plt.figure()
top_companies.plot(kind='bar')
plt.title("Average Price by Top Companies")
plt.xlabel("Company")
plt.ylabel("Average Price")
plt.xticks(rotation=45)
plt.show()
```

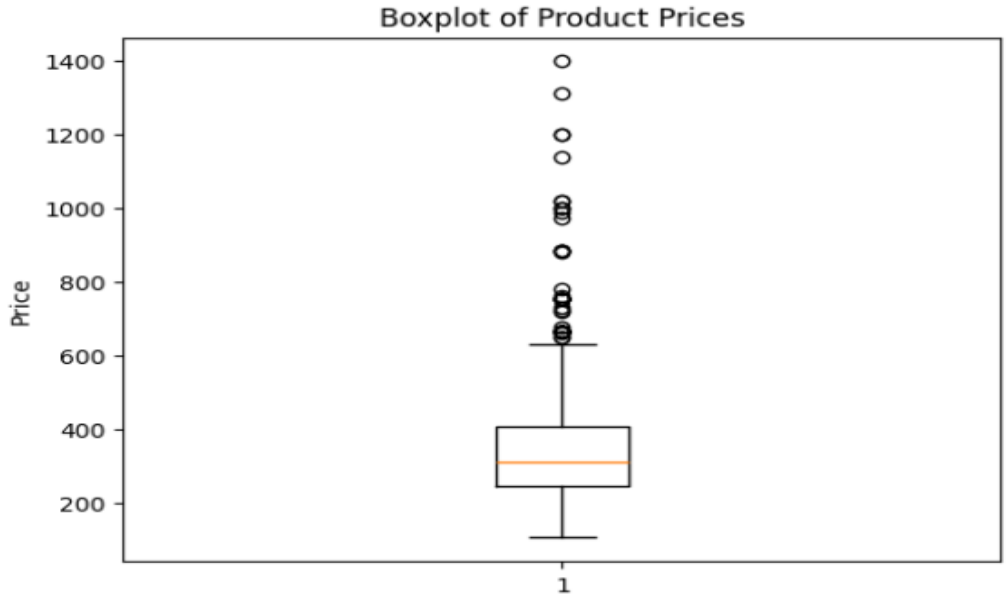


[55]:
```
page_trend = df.groupby('page')['price'].mean()

plt.figure()
page_trend.plot(kind='line')
plt.title("Average Price Trend by Page")
plt.xlabel("Page Number")
plt.ylabel("Average Price")
plt.show()
```

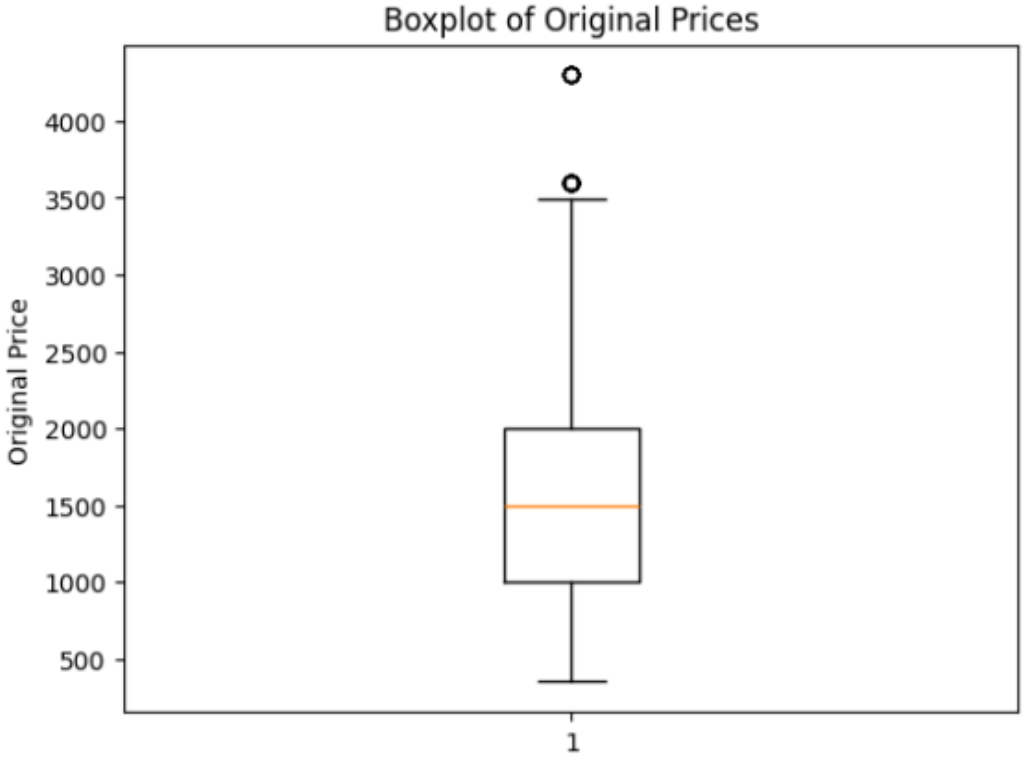
[56]:
```
plt.figure()
plt.scatter(df['price'], df['discount_pct'])
plt.title("Price vs Discount Percentage")
plt.xlabel("Price")
plt.ylabel("Discount %")
plt.show()
```



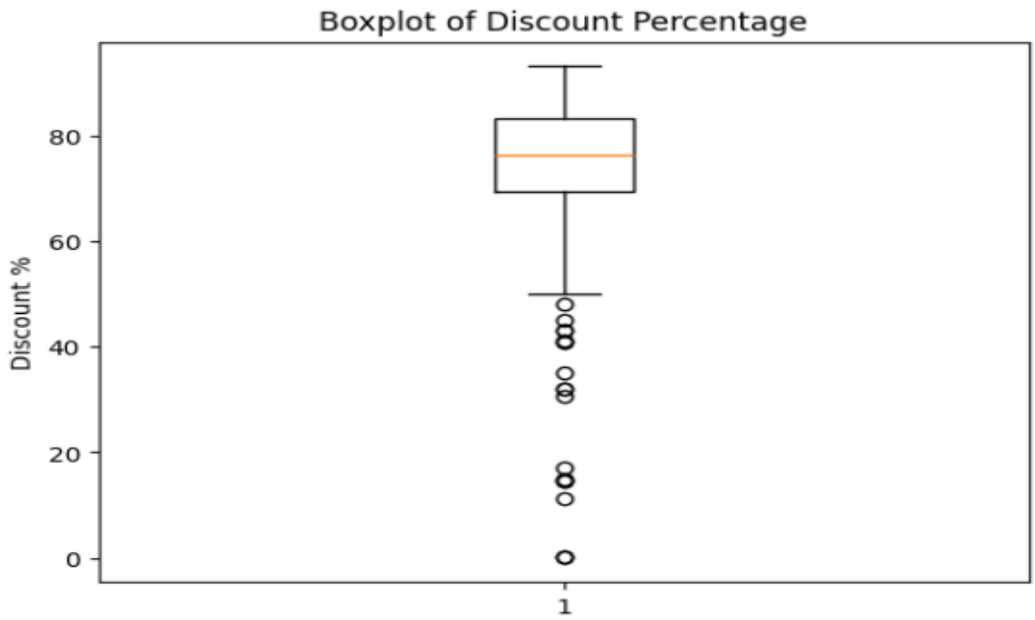
[57]:
```
import matplotlib.pyplot as plt

plt.figure()
plt.boxplot(df['price'])
plt.title("Boxplot of Product Prices")
plt.ylabel("Price")
plt.show()
```



[58]:
```
plt.figure()
plt.boxplot(df['original_price'])
plt.title("Boxplot of Original Prices")
plt.ylabel("Original Price")
plt.show()
```

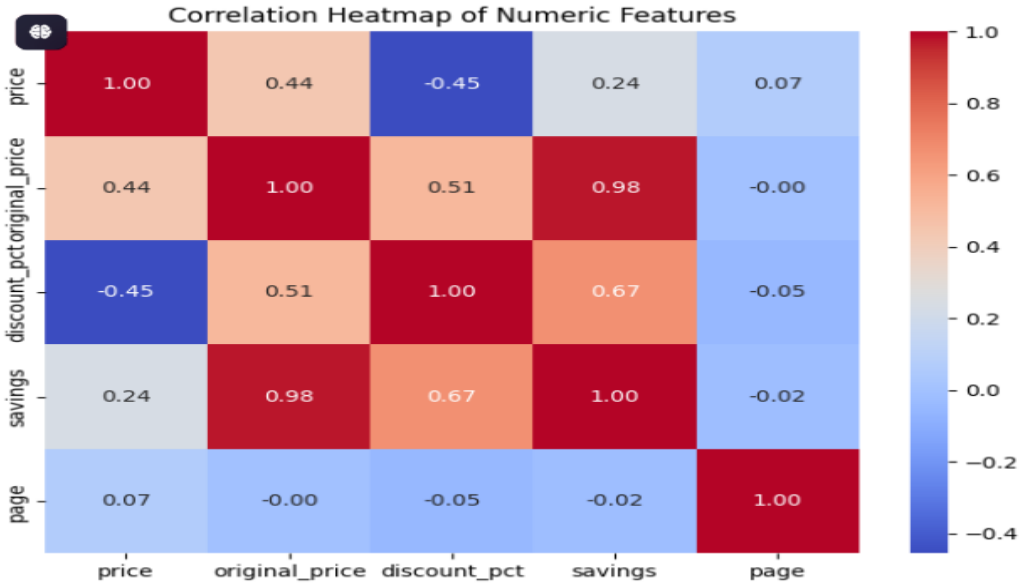


[59]:
```
plt.figure()
plt.boxplot(df['discount_pct'])
plt.title("Boxplot of Discount Percentage")
plt.ylabel("Discount %")
plt.show()
```



[60]:
```
plt.figure()
plt.boxplot([df['price'], df['original_price'], df['discount_pct']])
plt.title("Price vs Original Price vs Discount%")
plt.xticks([1,2,3], ['Price','Original Price','Discount %'])
plt.show()
```

```
[61]: import seaborn as sns
      import matplotlib.pyplot as plt

[62]: numeric_cols = ['price', 'original_price', 'discount_pct', 'savings', 'page']

[63]: corr_matrix = df[numeric_cols].corr()

[64]: plt.figure(figsize=(8,6))
      sns.heatmap(corr_matrix, annot=True, fmt=".2f", cmap="coolwarm")
      plt.title("Correlation Heatmap of Numeric Features")
      plt.show()
```



OUTPUT

```
[ ]: # The EDA of the Flipkart men's topwear dataset shows that the marketplace is strongly driven by a value-for-money and discount
     # -focused strategy rather than premium pricing. The average selling price is around ₹347, while the median is ₹310, indicating
     # most products are in the affordable range, with only a few higher-priced items pulling the average up. In contrast, the mean original
     # price is about ₹1560, creating a large gap between listed and selling prices, which highlights how heavily the platform depends on
     # discount-based pricing. With an average discount of over 75% and average customer savings of roughly ₹1213 per product, the data
     # clearly suggests that perceived savings play a major role in influencing purchase decisions. Visualizations further show that
     # products cluster in mid-price bands and that high-discount items are common, indicating that deals are used as key marketing
     # tools to attract customers. Overall, customer behavior in this segment is driven more by visible discounts and savings than by brand prestige,
     # making promotional pricing the central competitive strategy.
```

❖Output

❖ The EDA of the Flipkart men's topwear dataset shows that the marketplace is strongly driven by a value-for-money and discount-focused strategy rather than premium pricing. The average selling price is around ₹347, while the median is ₹310, indicating most products are in the affordable range, with only a few higher-priced items pulling the average up. In contrast, the mean original price is about ₹1560, creating a large gap between listed and selling prices, which highlights how heavily the platform depends on discount-based pricing. With an average discount of over 75% and average customer savings of roughly ₹1213 per product, the data clearly suggests that perceived savings play a major role in influencing purchase

decisions. Visualizations further show that products cluster in mid-price bands and that high-discount items are common, indicating that deals are used as key marketing tools to attract customers. Overall, customer behavior in this segment is driven more by visible discounts and savings than by brand prestige, making promotional pricing the central competitive strategy.

PHASE 4 : MACHINE LEARNING MODEL

PHASE 4 : MACHINE LEARNING MODEL

- Recommendation -> Suggest similar items

[65]:
```
features = ['name', 'company', 'price_band', 'discount_pct']
df_model = df[features].copy()
```

[66]:
```
df_model['discount_pct'] = df_model['discount_pct'].astype(str)
```

[67]:
```
df_model['combined_features'] = (
    df_model['name'] + " " +
    df_model['company'] + " " +
    df_model['price_band'] + " " +
    df_model['discount_pct']
)
```

[68]:
```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(stop_words='english')
feature_matrix = vectorizer.fit_transform(df_model['combined_features'])
```

[69]:
```
from sklearn.metrics.pairwise import cosine_similarity

similarity_matrix = cosine_similarity(feature_matrix)
```

[70]:
```python
def recommend(query, top_n=5):
    query = str(query).lower()

    # Search across multiple columns
    mask = (
        df['name'].astype(str).str.lower().str.contains(query) |
        df['company'].astype(str).str.lower().str.contains(query) |
        df['price'].astype(str).str.contains(query) |
        df['discount_pct'].astype(str).str.contains(query)
    )

    indices = df[mask].index

    if len(indices) == 0:
        return "Item not found!"

    idx = indices[0]

    scores = list(enumerate(similarity_matrix[idx]))
    scores = sorted(scores, key=lambda x: x[1], reverse=True)[1:top_n+1]

    product_indices = [i[0] for i in scores]

    return df[['name','company','price','discount_pct']].iloc[product_indices
```

[71]:
```python
unique_price = df.price.unique()
```

[72]:
```python
df.price.unique()
```

[72]:
```
array([ 298.,  338.,  459.,  306.,  192.,  285.,  210.,  237.,  266.,
        260.,  270.,  594.,  350.,  393.,  256.,  241.,  392.,  262.,
        416.,  318.,  226.,  252.,  220.,  446.,  998.,  322.,  243.,
        236.,  163.,  240.,  475.,  276.,  752.,  148.,  394.,  610.,
        267.,  234.,  431.,  313.,  219.,  328.,  235.,  263.,  337.,
        385.,  421.,  336.,  665.,  319.,  229.,  374.,  246.,  214.,
        211.,  289.,  212.,  255.,  265.,  420.,  282.,  367.,  250.,
        218.,  305.,  205.,  172.,  360.,  249.,  290.,  283.,  629.,
        295.,  291.,  280.,  442.,  457.,  277.,  507.,  608.,  354.,
        300.,  499.,  315.,  434.,  399.,  273.,  479.,  253.,  368.,
        180.,  232.,  228.,  120.,  248.,  330.,  329.,  391.,  452.,
        261.,  317.,  759.,  224.,  202.,  242.,  178.,  437.,  884.,
        175.,  279.,  372.,  388.,  436.,  308.,  207.,  450.,  410.,
        358.,  221.,  223.,  183.,  389.,  222.,  258.,  559.,  435.,
        662.,  440.,  443.,  879.,  370.,  233.,  427.,  209.,  257.,
        199.,  386.,  160.,  339.,  390.,  301.,  362.,  225.,  325.,
        287.,  402.,  883.,  340.,  408.,  185.,  478.,  213.,  288.,
        381.,  158.,  989.,  406.,  519.,  361.,  227.,  387.,  216.,
        404.,  278.,  269., 1139.,  558.,  162.,  649.,  344., 1310.,
```

[74]:
```python
recommend("Men Regular Fit Checkered Button Down Collar C")
```

[74]:

	name	company	price	discount_pct
834	Men Regular Fit Checkered Button Down Collar C...	RED TAPE	610.0	81.51
34	Men Regular Fit Solid Button Down Collar Casua...	RED TAPE	752.0	82.51
157	Men Regular Fit Solid Button Down Collar Casua...	RED TAPE	752.0	82.51
197	Men Regular Fit Solid Button Down Collar Casua...	RED TAPE	752.0	82.51
437	Men Regular Fit Solid Button Down Collar Casua...	RED TAPE	752.0	82.51

[75]:
```python
recommend("319")
```

[75]:

	name	company	price	discount_pct
122	Men Regular Fit Solid Spread Collar Casual Shirt	METRONAUT PLUS	457.0	83.07
592	Men Solid Round Neck Cotton Blend Black T-Shirt	BLIVE	223.0	77.68
99	Men Printed Round Neck Pure Cotton Multicolor ...	Magneto	305.0	66.07
139	Men Printed Round Neck Pure Cotton Multicolor ...	Magneto	305.0	66.07
942	Men Printed Round Neck Pure Cotton Multicolor ...	Magneto	309.0	69.07

[76]:
```python
recommend("7.9")
```

[76]:

	name	company	price	discount_pct
136	Men Solid Mandarin Collar Cotton Blend Maroon ...	TRIPR	241.0	87.94
177	Men Solid Mandarin Collar Cotton Blend Maroon ...	TRIPR	241.0	87.94
417	Men Solid Mandarin Collar Cotton Blend Maroon ...	TRIPR	241.0	87.94
776	Men Solid Mandarin Collar Cotton Blend Maroon ...	TRIPR	241.0	87.94
814	Men Solid Mandarin Collar Cotton Blend Maroon ...	TRIPR	241.0	87.94

[77]:
```python
recommend("Try This")
```

[77]:

	name	company	price	discount_pct
95	Men Striped Zip Neck Cotton Blend Brown T-Shirt	Try This	306.0	69.37
838	Men Checkered Zip Neck Cotton Blend White, Bla...	Try This	285.0	71.47
919	Men Colorblock Zip Neck Cotton Blend Black T-S...	Try This	240.0	75.98

[78]:
```
df.company.unique()
```

[78]:
```
<ArrowStringArray>
[                    'KAJARU',                      'TRIPR',
                  'Try This',                      'Surhi',
                    'VeBNoR',                    'N AND J',
                 'JACKBELLA',                      'DIMMY',
                   'Raymond',                      'WOXEN',
 ...
                 'Southcity',                 'Tee Gallery',
                      'DBAX',                     'GLAUCUS',
 'United Colors of Benetton',                  'BLUE SQUAD',
               'PrintHolic',                    'Lux Nitro',
               'hkCreation',                    'BlueFicus']
Length: 190, dtype: str
```

Train–Test Split -> Model Training -> Evaluation (Accuracy/RMSE/R^2)

[79]:
```
from sklearn.model_selection import train_test_split

X = df[['original_price', 'discount_pct', 'company', 'page']]
y = df['price']
```

[80]:
```
X = pd.get_dummies(X, columns=['company'], drop_first=True)
```

[81]:
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

[82]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
```

[82]:
```
LinearRegression
LinearRegression()
```

[83]:
```
y_pred = model.predict(X_test)
```

[84]:
```
from sklearn.metrics import mean_squared_error, r2_score

rmse = mean_squared_error(y_test, y_pred, squared=False)
r2 = r2_score(y_test, y_pred)

print("RMSE:", rmse)
print("R² Score:", r2)
```

```
RMSE: 29.968378939548487
R² Score: 0.9339899426650998
C:\Users\TRIDEV PAL\Downloads\anacondajupiter\Lib\site-packag
4 and will be removed in 1.6. To calculate the root mean squa
  warnings.warn(
```

[108]:
```
train_score = model.score(X_train, y_train)
print("Training Score:", train_score)
```

```
Training Score: 0.9632903094507861
```

[109]:
```
test_score = model.score(X_test, y_test)
print("Testing Score:", test_score)
```

```
Testing Score: 0.9339899426650998
```

[85]:
```
results = pd.DataFrame({
    'Actual Price': y_test.values,
    'Predicted Price': y_pred
})

results.head(10)
```

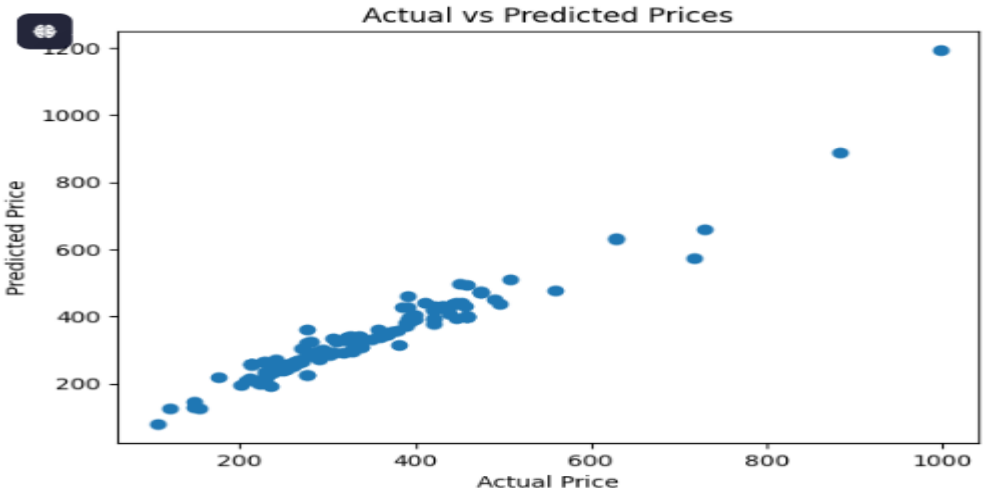
[85]:

	Actual Price	Predicted Price
0	459.0	399.856464
1	392.0	387.401691
2	393.0	394.705391
3	393.0	394.165069
4	393.0	392.544103
5	382.0	316.031325
6	475.0	473.591820
7	276.0	361.583190
8	279.0	282.668245

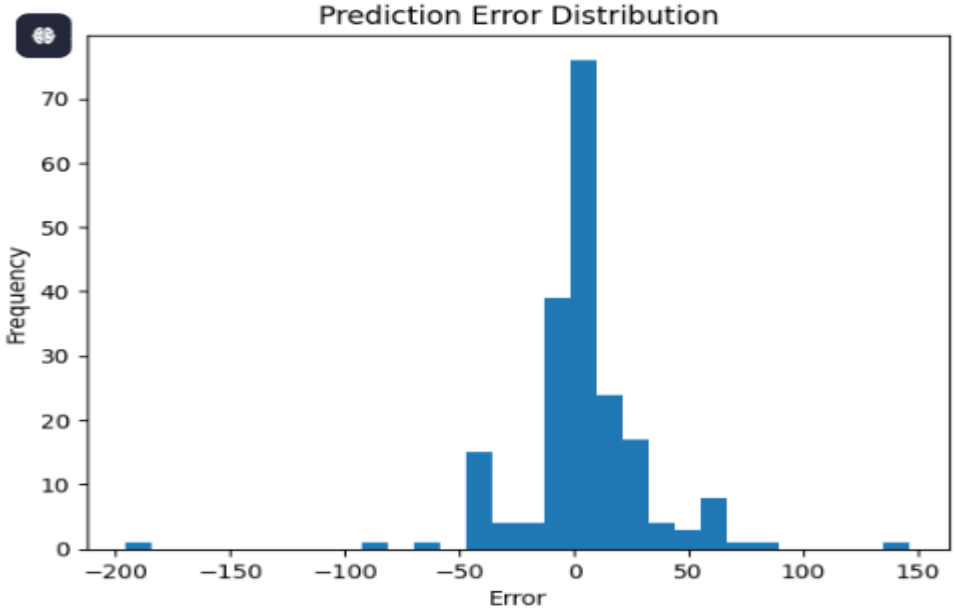
[86]:
```
print("RMSE:", rmse)
print("R² Score:", r2)
```

```
RMSE: 29.968378939548487
R² Score: 0.9339899426650998
```

[87]:
```
import matplotlib.pyplot as plt

plt.figure()
plt.scatter(y_test, y_pred)
plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Prices")
plt.show()
```



[88]:
```
errors = y_test - y_pred

plt.figure()
plt.hist(errors, bins=30)
plt.title("Prediction Error Distribution")
plt.xlabel("Error")
plt.ylabel("Frequency")
plt.show()
```



[89]:
```
coefficients = pd.DataFrame({
    'Feature': X.columns,
    'Coefficient': model.coef_
}).sort_values(by='Coefficient', ascending=False)

coefficients.head(10)
```

[89]:

	Feature	Coefficient
172	company_WROGN	348.203592
152	company_U.S. POLO ASSN.	301.046858
116	company_Raymond	210.876294
128	company_Snitch	195.550954
160	company_VAN HEUSEN	164.548595
77	company_LOUIS PHILIPPE	144.210243
136	company_THE BEAR HOUSE	137.952556
16	company_Allen Cooper	133.468903
53	company_HERE&NOW	129.247473
99	company_PARK AVENUE	122.303218

[90]:
```
# The high R² score indicates strong linear dependency between price
# and features such as original price and discount, which is expected in pricing models.
```

[91]:
```
df.head()
```

[91]:

	name	company	price	original_price	discount	offers	url	page	discount_pct	savings	price_band	high_discount
0	Men Solid Zip Neck Cotton Blend Black T-Shirt	KAJARU	298.0	999.0	70.0	Hot Deal	https://www.flipkart.com/kajaru-solid-men-zip-...	1	70.17	701.0	Budget	1
1	Men Printed Polo Neck Cotton Blend Black, Grey...	TRIPR	338.0	999.0	66.0	Buy 2 items, save extra 5%	https://www.flipkart.com/tripr-printed-men-pol...	1	66.17	661.0	Budget	1
2	Pack of 4 Men Solid Round Neck Cotton Blend Mu...	TRIPR	459.0	1999.0	77.0	Buy 2 items, save extra 5%	https://www.flipkart.com/tripr-solid-men-round...	1	77.04	1540.0	Budget	1
3	Men Striped Zip Neck Cotton Blend Black, White...	Try This	306.0	999.0	69.0	Only few left	https://www.flipkart.com/try-striped-men-zip-n...	1	69.37	693.0	Budget	1
4	Men Solid Henley Neck Cotton Blend Black T-Shirt	TRIPR	192.0	1999.0	90.0	Hot Deal	https://www.flipkart.com/tripr-solid-men-henle...	1	90.40	1807.0	Budget	1

[92]:
```
df.high_discount.to_csv("df.high_discount.csv",index=False)
```

[93]:
```
df.discount_pct.to_csv('df.discount_pct.csv',index=False)
```

[94]:
```
df.price_band.to_csv('df.df.price_band.csv',index=False)
```

[95]:
```
df.savings.to_csv('df.savings.csv',index=False)
```

[96]:
```
company_savings.to_csv('company_savings.csv')
```

[97]:
```
company_discount_pact.to_csv('df.company_discount_pact.csv',index=False)
```

[98]:
```
pd.DataFrame(unique_price).to_csv('unique_price.csv')
```

[99]:
```
results.to_csv('results.csv', index=False)
```

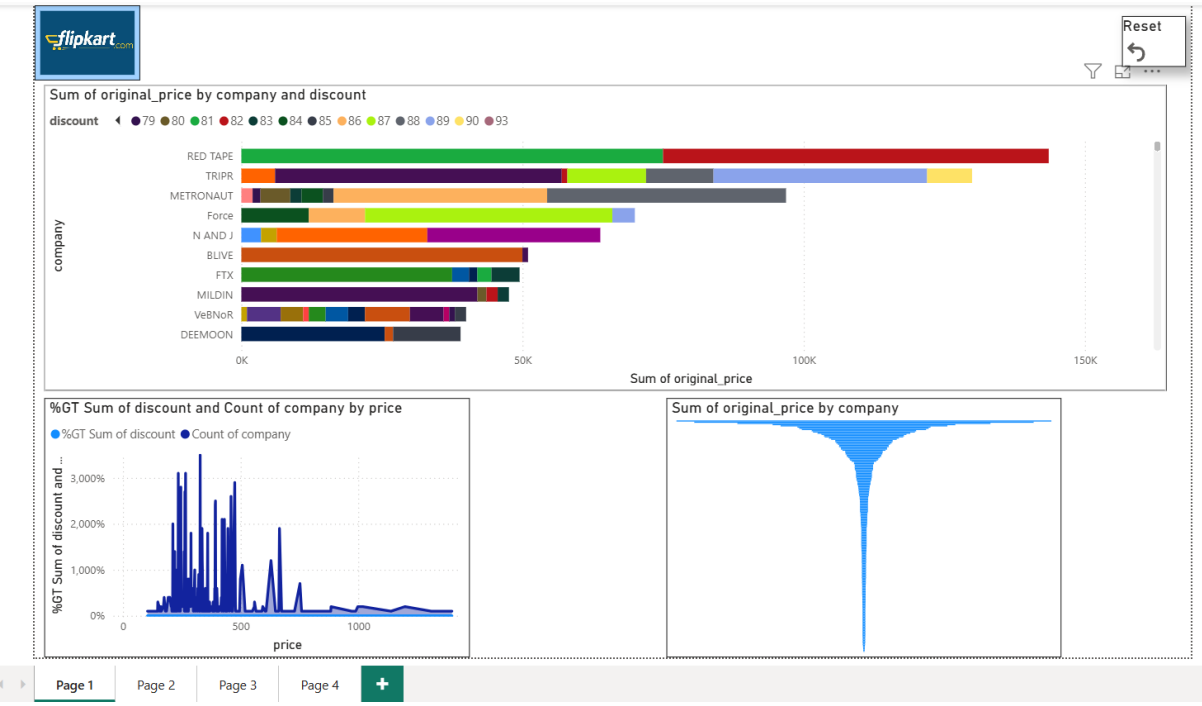
[100]:
```
coefficients.to_csv("coefficients.csv", index=False)
```

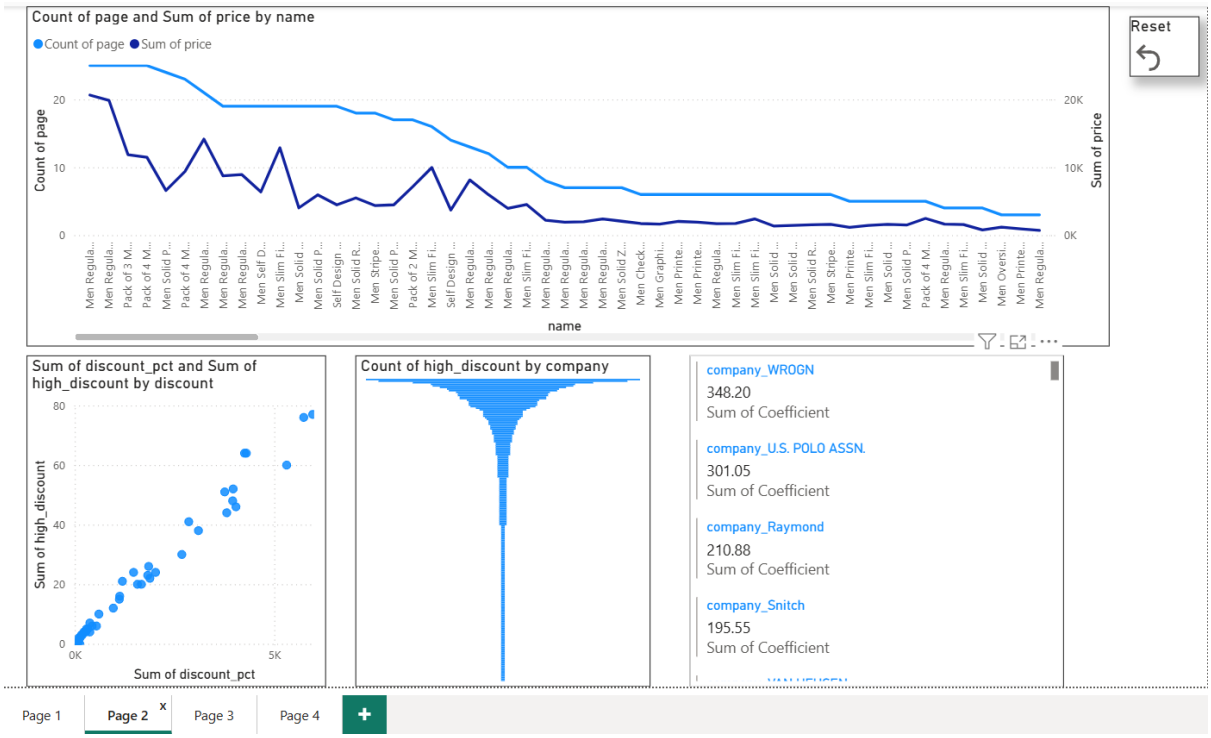
[101]:
```
df.to_csv("df.csv",index=False)
```

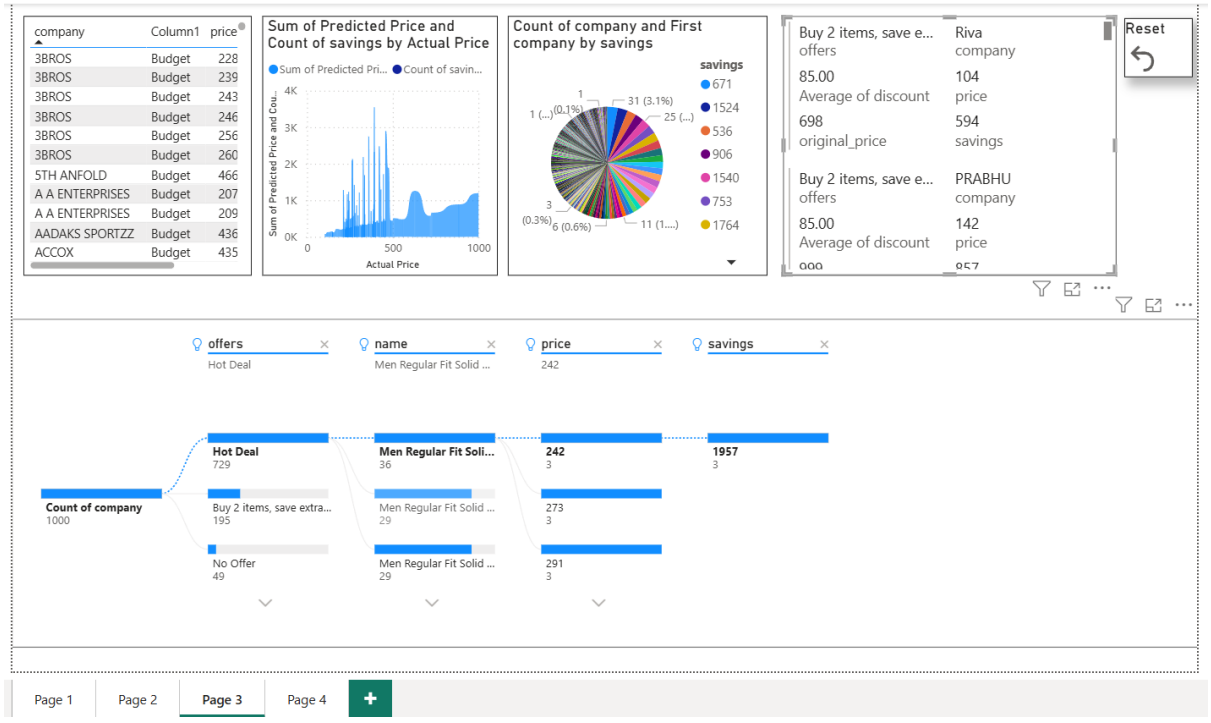
[ ]:

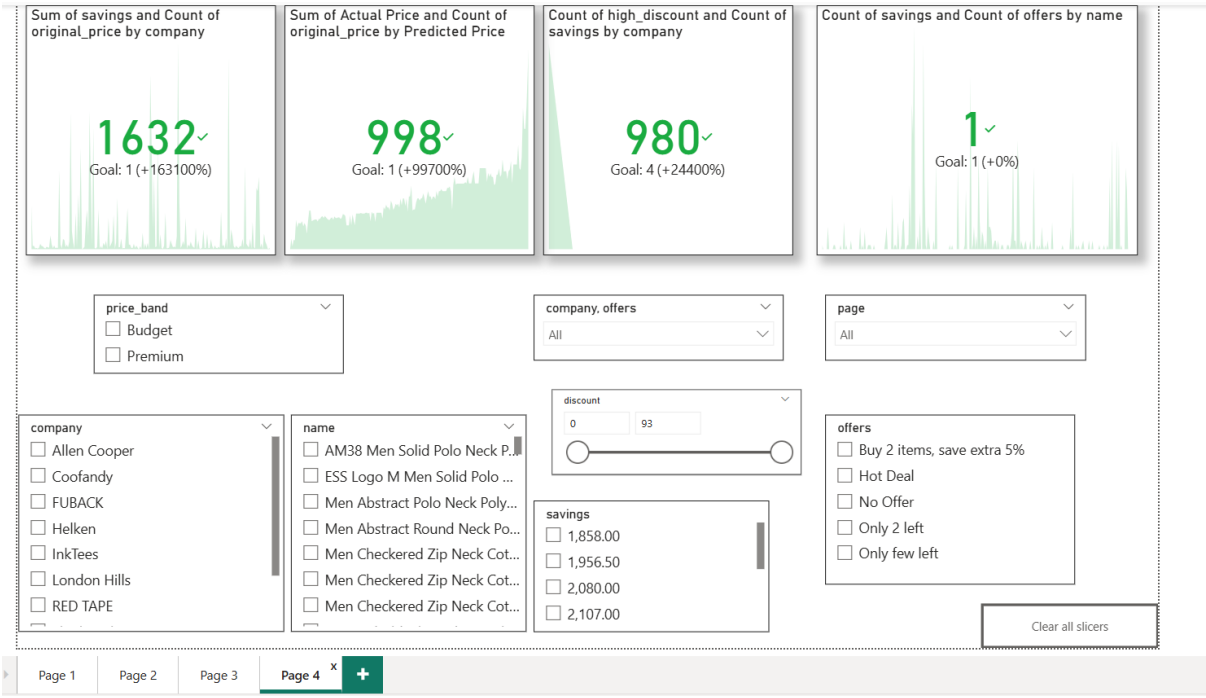
3. Model Performance
    - My model is performing well. It has a r^2 of 0.9339 and a RMSE of 29.9683 for Training score it is 0.9632 and for Testing score it is 0.93398.
      The higher the r^2 indicates the stronger linear dependency between price and feature such as original price and discount which is expected in a price predicting model.

PHASE 5 : INTERACTIVE DASHBOARD




Count of page and Sum of price by name
Count of page
Sum of price
Count of page
Sum of price
name
Reset
Sum of discount_pct and Sum of high_discount by discount
Sum of high_discount
Sum of discount_pct
Count of high_discount by company
company_WROGN
348.20
Sum of Coefficient
company_U.S. POLO ASSN.
301.05
Sum of Coefficient
company_Raymond
210.88
Sum of Coefficient
company_Snitch
195.55
Sum of Coefficient
Page 1
Page 2
Page 3
Page 4


company
Column1
price
3BROS Budget 228
3BROS Budget 239
3BROS Budget 243
3BROS Budget 246
3BROS Budget 256
3BROS Budget 260
5TH ANFOLD Budget 466
A A ENTERPRISES Budget 207
A A ENTERPRISES Budget 209
AADAKS SPORTZZ Budget 436
ACCOX Budget 435
Sum of Predicted Price and Count of savings by Actual Price
Sum of Predicted Price and Cou...
Actual Price
Count of company and First company by savings
savings
671
1524
536
906
1540
753
1764
Buy 2 items, save e...
offers
85.00
Average of discount
698
original_price
Riva
company
104
price
594
savings
Buy 2 items, save e...
offers
85.00
Average of discount
PRABHU
company
142
price
Reset
offers
Hot Deal
name
Men Regular Fit Solid ...
price
242
savings
Count of company
1000
Hot Deal
729
Buy 2 items, save extra...
195
No Offer
49
Men Regular Fit Soli...
36
Men Regular Fit Solid ...
29
Men Regular Fit Solid ...
29
242
3
273
3
291
3
1957
3
Page 1
Page 2
Page 3
Page 4



4. Key Business Insight
   - Customer purchase behaviour in men's fashion is driven by discount dept, midrange pricing and promotional offers rather than brand premium position.

5. Recommendations
   - Our platform should flow a promotional driven, value focused strategy where revenue comes from discount structure ,midrange pricing

and mult offers building rather than premium position.